



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez.

Asignatura: Programación orientada a objetos.

Grupo: 07

No de Práctica(s): 5

Integrante(s): 323331229

426060019

323217156

312109239

323195386

321321976

*No. de lista o
brigada:* 6

Semestre: 3^{ro}

Fecha de entrega: 02/10/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	3
1.1. Planteamiento del problema	3
1.2. Motivación	3
1.3. Objetivos	3
2. Marco Teórico	4
3. Desarrollo	5
3.1. Ejercicio 1.	5
3.2. Ejercicio 2.	6
3.3. Ejercicio 3.	8
4. Resultados	10
4.1. Resultados del Ejercicio 1.	10
4.2. Resultados del Ejercicio 2.	11
4.3. Resultados del Ejercicio 3.	11
5. Conclusiones	12

1. Introducción

1.1. Planteamiento del problema

Para esta practica se plantean los siguientes ejercicios: En el ejercicio 1, debemos preguntarle a nuestro LLM si es conveniente usar métodos modificadores en los constructores, justificando la respuesta en al menos media cuartilla.

Para el Ejercicio 2, nos piden crear una clase llamada Empleado con los atributos de tipo String para nombre y apellido, y de tipo double para el salario mensual, incluyendo un constructor para inicializarlos y métodos modificadores, validando que el salario no se establezca si no es un valor positivo. Crear una clase de prueba MainEmpleado para instanciar dos objetos, mostrar el salario anual de cada uno, aplicarles un aumento del 10% y volver a mostrar su salario anual actualizado.

Finalmente, en el Ejercicio 3 tenemos que crear una clase Fecha con tres atributos enteros (mes, día y año), un constructor que asuma valores correctos, métodos modificadores, un método mostrarFecha que imprima los datos separados por una diagonal ("/"), y una clase MainFecha que ponga a prueba la clase anterior.

1.2. Motivación

El objetivo de esta práctica es mostrar cómo el encapsulamiento limita el acceso directo a los datos de un objeto mediante los modificadores de visibilidad, obligando al sistema a emplear métodos controlados de acceso que aseguren que los datos ingresados y alterados conserven siempre una condición coherente y segura dentro del código.

1.3. Objetivos

- **General:** Aplicar el concepto de Abstracción para el diseño de clases que conforman una solución, usando el encapsulamiento para proteger la información y ocultar la implementación.
- **Específicos:**
 - Analizar y justificar de forma teórica la pertinencia de utilizar métodos modificadores de acceso directamente dentro de los métodos constructores de una clase.
 - Implementar el encapsulamiento la clase Empleado, protegiendo sus atributos y aplicando reglas de validación en sus métodos (asegurar salarios positivos).
 - Simular el comportamiento de los objetos instanciados mediante una clase principal (Main), aplicando operaciones de lectura y modificación de estado (como el aumento de salario del 10 %).
 - Desarrollar una clase Fecha que restrinja el acceso a sus componentes (día, mes, año) y exponga su estado únicamente a través de métodos de formato estructurado.

2. Marco Teórico

1. **Abstracción:** La abstracción es el proceso de identificar las características (atributos) y acciones o comportamientos (métodos) propios de un elemento que deseamos representar. [1]
2. **Objeto:** Representan cosas del mundo real, los elementos de un objeto se dividen en dos categorías principales atributos y métodos. [4]
3. **Encapsulamiento:** Sucede cuando algo es envuelto en una capa protectora. Cuando el encapsulamiento se aplica a los objetos, significa que los datos del objeto están protegidos, “ocultos” dentro del objeto, el resto del programa no puede acceder de manera directa a los datos de un objeto lo tiene que hacer con ayuda de los métodos del objeto. [2]
4. **Modificadores de acceso:** Los modificadores de acceso (o de visibilidad) establecen permisos y nivel de accesibilidad de los miembros (atributos y métodos) de una clase hacia otras partes del programa, lo que permite proteger los datos y aplicar el principio de encapsulamiento. [3]
5. **Public (Público):** El miembro es accesible desde cualquier otra clase u objeto en todo el sistema. [3]
6. **Protected (Protegido):** El miembro es accesible para la propia clase, para clases en el mismo paquete y fundamentalmente para sus clases hijas (subclases) mediante la herencia. [3]
7. **Private (Privado):** El miembro solo es visible desde dentro de la propia clase que lo define. Oculta el dato o método al resto del programa. [3]
8. **Métodos de acceso:** Son el medio para acceder a los atributos privados de un objeto y manipularlos. Estos métodos, generalmente llamados getters (para leer) y setters (para modificar). [2]

3. Desarrollo

3.1. Ejercicio 1.

Este ejercicio consistió en colaborar con nuestro LLM preguntándole: ¿Es conveniente usar métodos modificadores en los constructores? Con esta pregunta Gemini nos respondió lo siguiente:

- El uso de métodos modificadores dentro de los constructores es una mala práctica de diseño en la programación orientada a objetos debido al grave riesgo de corromper la integridad de la herencia. El argumento principal radica en el peligro del polimorfismo durante la fase de instanciación. Si un constructor invoca un método modificador que es público o protegido, cualquier clase hija tiene la capacidad de sobrescribir dicho método. Al crear una instancia de la clase hija, el sistema ejecuta primero el constructor de la clase padre; este llamará a la versión sobrescrita del modificador antes de que la clase hija haya tenido la oportunidad de inicializar sus propias variables. Esto provoca que el método opere sobre un objeto *“a medio construir”*, lo que genera comportamientos erráticos o caídas del sistema difíciles de rastrear.

Esta práctica distorsiona el ciclo de vida natural del objeto. El propósito exclusivo de un constructor es cimentar un estado inicial sólido y válido, asignando valores directamente a las propiedades. Por el contrario, los modificadores existen para alterar el estado de un objeto que ya está completamente vivo y funcional en el sistema. Utilizarlos durante la construcción asume erróneamente que el objeto ya está listo para mutar antes de haber terminado de nacer.

Finalmente, depender de los modificadores en la inicialización fomenta arquitecturas frágiles y dificulta la creación de objetos inmutables, los cuales son fundamentales para garantizar la seguridad en aplicaciones complejas. Si la intención original era no repetir la lógica de validación de datos, la salida elegante y segura siempre será extraer esas reglas a métodos privados. De esta forma, blindas la lógica interna, previenes que agentes externos o clases derivadas interfieran en el proceso de creación y garantizas que cada objeto llegue al sistema en un estado impecable.

Pero la respuesta de Claude fue muy diferente, indicando que si es conveniente:

- Sí es conveniente, pero con una condición: que el método modificador no pueda ser sobrescrito por una subclase.

El principal argumento a favor es evitar duplicar lógica. Si el setter ya valida o normaliza el dato, reutilizarlo en el constructor garantiza que el objeto nunca nazca en un estado inválido y que las reglas de negocio vivan en un solo lugar. Así, si la validación cambia, solo hay que modificarla una vez.

El argumento en contra es el riesgo de la herencia. Cuando el constructor de la clase base llama a un método sobrescribible, ese método se ejecuta antes de que

la subclase haya inicializado sus propios campos. El resultado son errores sutiles: atributos nulos o en cero, comportamientos inesperados y fallas difíciles de depurar, porque el código parece correcto a simple vista.

Por eso, la práctica recomendable es hacer que el método sea privado o final (o que la clase no sea heredable). Con eso se conservan los beneficios de reutilizar la validación sin abrir la puerta al problema. Otra alternativa sana es separar la validación en un método privado auxiliar, usarlo tanto en el constructor como en el setter público, y así no depender de un método que otros puedan modificar.

También conviene preguntarse si el objeto realmente necesita setters. Si no debe cambiar después de creado, lo más sólido es hacerlo inmutable: validar todo en el constructor y no exponer modificadores.

3.2. Ejercicio 2.

Para este ejercicio se creó una clase llamada Empleado que guarda tres datos de cada persona: su nombre, su apellido y su salario mensual, una regla importante es que el salario solo se acepta si es un número positivo. Después se hizo una segunda clase, MainEmpleado, que sirve para probar todo lo anterior, en ella se crean dos empleados con sus propios datos y se muestra cuánto gana cada uno al año, que se calcula multiplicando su salario mensual por doce luego a ambos se les aplica un aumento del 10 % sobre su salario mensual, y finalmente se vuelve a mostrar su salario anual ya actualizado, para comprobar que el aumento se reflejó correctamente y comparar el antes y el después.

- a) Se crea la clase Empleado, esta actúa como la plantilla para cualquier empleado que exista en el programa.
- b) Declaramos tres atributos nombre, apellido y salarioMensual. Al usar la palabra `private` significa que bloqueamos el acceso directo desde el exterior; nadie puede leer o modificar estos datos.
- c) Creamos el constructor `public Empleado(...)`. Su lógica entra en acción en el instante exacto en que instanciamos un empleado nuevo en el programa, recibe los tres datos (nombre, apellido, salarioMensual) y utiliza la palabra clave **this** que significa "los atributos de este molde" para guardar esos valores recibidos dentro de los atributos privados que bloqueamos en el paso anterior.
- d) Como los atributos están escondidos por ser privados, creamos los métodos `get` su única función es cuando el resto del programa necesita saber cómo se llama el empleado o cuánto gana, llama a estos métodos, y estos simplemente le retornan (`return`) el valor actual.
- e) Creamos los métodos `set` para permitir que los datos cambien en el futuro (por ejemplo, si alguien se cambia el apellido) en `setSalarioMensual` en lugar de simplemente cambiar el dato, aplicamos un filtro lógico usando un `if/else`. Si el nuevo salario que intentan poner es mayor a 0, el cambio se autoriza y se guarda, si es 0 o negativo lanza una advertencia en la consola.

- f)* Creamos el método `aumentarSalario()` que toma la variable `salarioMensual`, le saca el 10% multiplicándola por 0.10, suma ese porcentaje al salario original, y finalmente reemplaza el valor viejo por este nuevo total.
- g)* Creamos el método `mostrar()` que toma pedazos de texto fijo (como "**Empleado:**") y los concatena con el valor exacto que tengan las variables en ese momento, mandando todo ese bloque de texto a la pantalla de la consola.
- h)* Creamos la clase `MainEmpleado` y dentro definimos el método `public static void main`.
- i)* Continuamos con la instanciación `Empleado empleado1 = new Empleado(...)`, la lógica es tomar el "molde" (la clase `Empleado`) y fabricar un objeto, usamos la palabra **new** para llamar a tu constructor y le inyectamos los datos iniciales.
- j)* Como los datos están protegidos como privados dentro del objeto, nos obligan a usar los getters (`getNombre()`, `getApellido()`, `getSalarioMensual()`), con ellos podemos extraer cada valor del objeto lo guardamos en variables temporales locales (`nombre1`, `apellido1`, `salario1`) y las imprimimos en la pantalla una por una. Finalmente, llamamos al método `empleado1.mostrar()` para que el objeto mismo imprima su resumen completo.
- k)* Con los setters, la lógica cambia de lectura a escritura, tomamos el objeto `empleado1` y le damos la orden de sobrescribir sus datos internos, le enviamos un nuevo salario el cual pasará por el filtro del `if` en la clase original y será aceptado.
- l)* Para comprobar que la mutación fue exitosa creamos nuevas variables (`nombre2`, etc.), le preguntamos al objeto cuáles son sus datos actuales usando los getters y los imprime, al llamar otra vez a `empleado1.mostrar()`, la consola reflejará la nueva identidad del empleado y su nuevo sueldo base.
- m)* Llamamos al método `empleado1.aumentarSalario`, el objeto toma su salario actual le calcula el 10% internamente y se actualiza a sí mismo, Imprime un texto ("**Aumento Salario (10\%)**") como separador visual y llamamos por última vez a `empleado1.mostrar()`. La lógica de este último paso es simplemente visualizar el resultado final después de la operación matemática, donde se vera el salario ya incrementado.

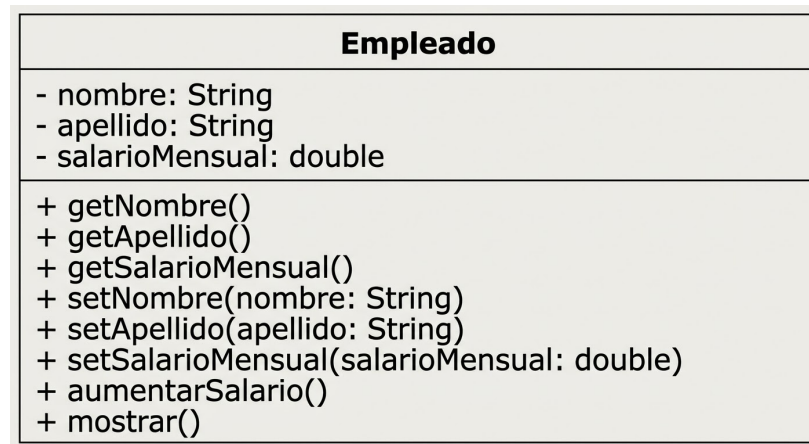


Figura 1: Diagrama UML.

Como prueba se ingresan los datos iniciales del empleado ("Valeria", "Torres", salario de 1200) y posteriormente se introducen nuevos valores para actualizarlo ("Andrés", "Hernández", salario de 2000).

3.3. Ejercicio 3.

Para este ejercicio se creó una clase llamada Fecha que guarda tres datos: el mes, el día y el año, al crear una fecha se le asignan estos tres valores desde el inicio, y se parte de la idea de que quien la crea ya escribió valores correctos, tiene un método llamado `mostrarFecha`, que se encarga de imprimir la fecha en pantalla con sus tres partes separadas por una diagonal, por ejemplo mes/día/año.

Por último, se hizo una clase `MainFecha` que sirve para probar todo lo anterior. En ella se crea una fecha con valores de ejemplo, se muestra en pantalla, se le cambian algunos de sus datos con los métodos modificadores y se vuelve a mostrar, para comprobar que los cambios se aplicaron bien y que el formato con diagonales se mantiene.

- Definimos la clase Fecha, que será la plantilla para crear y manejar fechas en el programa.
- Declaramos las variables enteras mes, día y año como privadas (`private`) para bloquear el acceso directo.
- Inicializamos el constructor `Fecha(...)` el cual entra en acción justo cuando creamos una nueva fecha, su función es recibir los tres números iniciales y guardarlos en las variables de la clase usando `this`.
- Como encapsulamos las variables creamos los métodos `get` que funcionan como un "modo lectura" solo devuelven el número que tienen guardado cuando el resto del programa lo solicita.
- Creamos los métodos `set` para permitir que el día, mes o año se cambien en el futuro.

- f) Creamos el método `mostrarFecha()` que toma los tres números enteros, los convierte a texto y les pega el símbolo / en medio para imprimirlos en la pantalla con el formato estadounidense: `mes/dia/año`.
- g) Importamos la herramienta `Scanner` para poder leer lo que el usuario teclee en la consola, además declaramos variables temporales (de texto y de número) para ir guardando lo que escriban.
- h) Te pide el día, mes y año uno por uno el `do-while` es para atrapar al usuario si presiona **"Enter"** sin escribir nada (`isEmpty()`), le tira un error y le vuelve a preguntar hasta que ponga algo, una vez que escribe un dato lo convierte de texto (`String`) a número real (`Integer.parseInt()`) para poder guardarlo.
- i) Toma esos tres números que el usuario acaba de escribir en la consola y los mete al **"molde"** creando el objeto `fecha` mediante `new Fecha(dia, mes, año)`.
- j) Le saca al objeto los datos que acaba de guardar usando los métodos `get`, los imprime uno por uno en la consola, y luego ejecuta `mostrarFecha()` para que el objeto se imprima solito en formato `mes/día/año`.
- k) Con los `Setters` olvida lo que el usuario ingresó y sobrescribe los datos del objeto directamente en el código. Le asigna el día 31, mes 7 y año 2019.
- l) Vuelve a hacer exactamente lo de sacar datos con `getters` e imprimir el formato para demostrar en pantalla que la fecha sí cambió. Al terminar, simplemente apaga el `Scanner` (`scanner.close()`) para liberar la memoria del sistema.

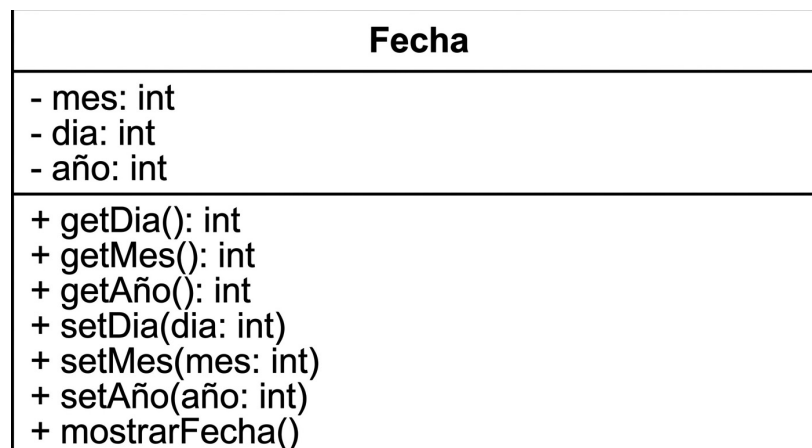


Figura 2: Diagrama UML.

Como prueba se ingresan por teclado los números correspondientes al día: 24, mes: 12 y año: 2006 y posteriormente se introducen directamente en el código los nuevos valores (31, 7, 2019) para modificar la fecha.

4. Resultados

4.1. Resultados del Ejercicio 1.

Se muestra la respuesta obtenida al consultar al LLM.

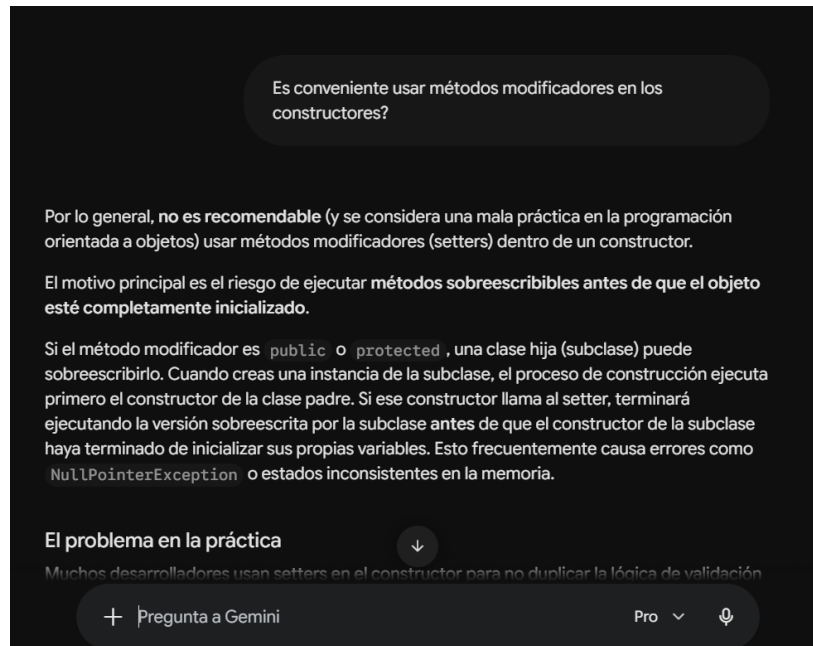


Figura 3: Respuesta de Gemini.



Figura 4: Respuesta de Claude.

4.2. Resultados del Ejercicio 2.

Se obtiene en pantalla la impresión de los datos originales, luego la confirmación de los datos modificados, y finalmente el registro del empleado mostrando su salario recalculado a 2200 tras aplicar el aumento del 10 %.

```
PS C:\Users\o-ale\OneDrive\Documentos\P00\Practica_5> javac *.java
PS C:\Users\o-ale\OneDrive\Documentos\P00\Practica_5> java MainEmpleado
Valeria
Torres
1200.0
Empleado: Valeria Torres salario anual $ 1200.0
Andrés
Hernández
2000.0
Empleado: Andrés Hernández salario anual $ 2000.0
Aumento Salario (10%)
Empleado: Andrés Hernández salario anual $ 2200.0
PS C:\Users\o-ale\OneDrive\Documentos\P00\Practica_5> █
```

Figura 5: Ejecución del código Clase Empleado.

4.3. Resultados del Ejercicio 3.

Se obtienen en pantalla las instrucciones pidiendo los datos, la impresión individual de los números ingresados, la fecha concatenada en formato mes/día/año, y finalmente la confirmación de la fecha modificada imprimiendo los nuevos valores tanto separados como en su formato final.

```
Ingresa un año: 2006
24
12
2006
12/24/2006
31
7
2019
7/31/2019
PS C:\Users\o-ale\OneDrive\Documentos\P00\Practica_5> █
```

Figura 6: Ejecución del código Clase Fecha.

5. Conclusiones

A través de esta práctica se comprobó la importancia del encapsulamiento en la Programación Orientada a Objetos como herramienta central para proteger el estado interno de las clases. El uso estructurado de modificadores y métodos de acceso permitió asegurar que los objetos instanciados, como Empleado o Fecha, mantengan siempre valores lógicos y consistentes frente a las interacciones de otras partes del programa.

Referencias

- [1] Abstracción en POO. (s. f.). <https://platzi.com>.<https://platzi.com/blog/abstraccion-en-poo-2/>
- [2] Olguín, Abstracción y encapsulamiento. Unidades de Apoyo Para el Aprendizaje - CUAED - UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3076/mod_resource/content/1/UAPA-Abstraccion-Encapsulamiento/index.html
- [3] Programador Certificado Java 2. (2.a ed.). (1992). Alfaomega Grupo Editor.
- [4] Sandoval, E. (2023, 21 septiembre). ¿Qué es un objeto en programación? Ebac. <https://ebac.mx/blog/objeto-en-programacion>
- [5] Sobrecarga de metodos y de constructores. (s.f.). http://profesores.fib.unam.mx/carlos/java/java_basico4_6.html